

Module 10 — Applied Lab: FastAPI Wrapping NLP (Natural Language Processing), Knowledge Graph, and RAG (Retrieval-Augmented Generation)

Due: Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

You will ship a containerized FastAPI backend that exposes three typed REST (Representational State Transfer) endpoints — one wrapping spaCy Named Entity Recognition (NER), one wrapping the Module 9 Week B deterministic NL—Cypher mapper against a pre-loaded recipe Neo4j graph, and one wrapping a RAG (Retrieval-Augmented Generation) pipeline against a Weaviate vector index of recipe-domain documents — plus a containerized Next.js frontend with three pages that consume those endpoints. The deliverable is the running backend image, the running frontend image, and a Lab-level Docker Compose file that brings the two services up together for end-to-end demonstration. The full four-service composition (with Neo4j + Weaviate as additional containers) is the Integration Task; the Lab ends at the two-service `api` + `web` Compose.

Setup

If you have not already installed the Module 10 packages, go back to the [Setup: install before the concept session](#) section of the reading and complete the install before proceeding.

Lab template repo: [LevelUp-Applied-A2/m10-110](#). Click “Use this template” —> “Create a new repository” on the template page, name your copy `m10-110`, leave *Visibility* as *Public*, and create the repository under your own GitHub account.

```
# After forking the template:
git clone https://github.com/<your-username>/m10-110.git
cd m10-110
git checkout -b lab-10-fastapi-kg-service
python -m venv .venv
source .venv/bin/activate # macOS / Linux
pip install -r api/requirements.txt
(cd web && npm ci)
```

The starter ships the W9B recipe Neo4j fixture (`api/seed.cypher`), the M8 chunked-docs RAG corpus (`api/seed_docs.json`), the vendored W9B deterministic mapper (`api/w9b_mapper/` — DO NOT MODIFY), the vendored M8 RAG generator cypher (`api/m8_rag/` — DO NOT MODIFY), the pre-implemented Weaviate seed script (`api/seed_weaviate.py` — DO NOT MODIFY), and a pre-scaffolded Next.js project (`web/`) with `package.json`, `package-lock.json`, `tsconfig.json`, `next.config.js`, and a landing page. Your job is to author the rest.

Overview

The Lab domain is **recipes** — a knowledge graph of recipes / ingredients / techniques (carry-forward from Module 9 Week B) plus a RAG corpus of recipe-domain documents (carry-forward from Module 8). The Day 2 Support Instructor demo uses a differentiated **movie information service** + **movie-review RAG** domain — same skills, different schema and function bodies. You cannot copy the demo into your submission.

You will implement six tasks across the backend and the frontend:

- NLP endpoint** — `POST /extract` wrapping spaCy NER.
- KG endpoint** — `POST /kg/query` wrapping the W9B deterministic mapper against Neo4j.
- RAG endpoint** — `POST /rag/answer` wrapping the M8 pipeline against Weaviate, with grounding refusal.
- Pydantic models** — request and response shapes for all three endpoints.
- FastAPI infrastructure** — lifespan opening Neo4j / Weaviate / spaCy / generator once; `/healthz` + `/readyz`; CORS (Cross-Origin Resource Sharing); structured errors; backend Dockerfile; Lab Docker Compose.
- Next.js frontend** — three TypeScript pages calling the three endpoints; hand-written TypeScript (TS) interfaces in `lib/types.ts`; multi-stage Node Dockerfile.

The next sections walk each task in turn.

Task 1 — `POST /extract` (NLP wrap)

Wrap the spaCy `en_core_web_sm` pipeline behind a typed endpoint.

Input contract: `{ "text": str }` with `text` non-empty and ≤ 5000 characters. **Output contract:** `{ "entities": [ { "text": str, "label": str, "start": int, "end": int } ] }` ordered by `start` ascending.

Files to author: `api/nlp.py` (load the pipeline, expose `extract_entities(text) -> list[Entity]`), `api/main.py` (the `/extract` path operation), `api/models.py` (`ExtractRequest`, `Entity`, `ExtractResponse`).

The pipeline must load once in `lifespan` and be reused per request. Loading inside the path-operation body causes a multi-second cold start on every request and intermittent test timeouts.

- Tests that will run:**
- `test_extract_returns_entities_ordered_by_start` — verifies the response is sorted by `start` ascending.
 - `test_extract_rejects_empty_text` — verifies 422 on empty `text`.
 - `test_extract_rejects_oversized_text` — verifies 422 on `text` > 5000 characters.

Task 2 — `POST /kg/query` (KG wrap)

Wrap the vendored Module 9 Week B deterministic mapper (`api/w9b_mapper/`).

Input contract: `{ "question": str }` with `question` non-empty and ≤ 500 characters. **Output contract:** `{ "cypher": str, "rows": list[dict], "count": int }` where `cypher` is the exact string emitted by the mapper, `rows = [record.data() for record in result]`, `count = len(rows)`. **Error contract:** the mapper's `UnsupportedQueryError` becomes HTTP 422 with structured detail `{ "reason": "unsupported_question", "supported_patterns": [...] }` — never let it bubble to 500.

Files to author: `api/kg.py` (the mapper-to-Cypher wrap), updates to `api/main.py`, `api/deps.py` (the `get_session()` dependency that yields a Neo4j session from the driver on `app.state`), updates to `api/models.py`.

The Neo4j Bolt driver must open once in `lifespan` and close on shutdown. The `get_session()` dependency yields a per-request session from the shared driver.

- Tests that will run:**
- `test_kg_query_returns_cypher_rows_count` — response has all three keys; `count == len(rows)`; `cypher` starts with `MATCH`.
 - `test_kg_query_unsupported_returns_422` — `UnsupportedQueryError` produces 422 with the `unsupported_question` reason and the list of supported patterns.

Task 3 — `POST /rag/answer` (RAG wrap with grounding)

This is the densest task. Wrap the four-stage RAG pipeline behind a typed endpoint whose contract enforces grounding.

Input contract: `{ "question": str, "k": int = 4 }` with `question` non-empty and ≤ 500 characters and `k` in `[1, 10]`. **Output contract:** `{ "answer": str, "citations": list[ { "chunk_id": int, "score": float } ], "score": float, "confidence": float }`.

Methodology (stated identically here, in `lab-spec.md`, and in the function docstrings):

- Retrieve** by externally encoding the question with `sentence-transformers` (`embedder.encode(question).tolist()`) and querying Weaviate with `with_near_vector: weaviate.query.get("Chunk", [{"text": "Chunk_ID"}].with_near_vector({"vector": vector}).with_additional({"distance":}).with_limit(k).do()`. The Weaviate class is configured `vectorizer=none` (we embed externally — see Reading Section 11 point 3), so `with_near_text` will fail at runtime with `KeyError: 'data'`. The same `sentence-transformers/all-MiniLM-L6-v2` model the seed used at ingest is loaded once at startup in `lifespan` and injected via `Depends(get_embedder)`.
- Assemble** a prompt with the retrieved chunks numbered `[1], [2], ...` Instructing the model to cite each claim with the source number in square brackets.
- Generate** with `flan-t5-base` (the vendored M8 generator), `max_new_tokens=256`, `do_sample=False` for determinism.
- Cite** — extract citation markers using the regex `\[(\d+)\]`; keep only markers in the `[1..k]` range; map back to `chunks[i]["chunk_id"]`.
- Grounding refusal:** if retrieval returns zero chunks **OR** the generated text has no valid citation markers, return the sentinel response: `{ "answer": "I cannot answer this from the available sources", "citations": [], "confidence": 0.0 }`.
- Confidence** is `mean(citation_scores)` where each citation's score is `1.0 - chunk.distance`, clipped to `[0, 1]`.

Validity constraint: every `chunk_id` in the response must correspond to a chunk Weaviate actually returned in the top-`k`.

Files to author: `api/rag.py` (the four-stage pipeline; the prompt template; the citation extractor), updates to `api/main.py`, `api/deps.py` (`get_weaviate()`, `get_generator()`, `get_embedder()`), updates to `api/models.py`.

The Weaviate client and the generator must load once in `lifespan`. `flan-t5-base` loads in ~30 seconds the first time; loading per request will time out your tests.

- Tests that will run:**
- `test_rag_answer_returns_grounded_answer` — seeded question returns 200 with `len(citations) >= 1`, `confidence > 0`, `answer` not the sentinel.
 - `test_rag_answer_empty_retrieval_returns_sentinel` — zero-match query returns 200 with the sentinel and `citations == []`.
 - `test_rag_answer_citation_ids_are_valid` — every cited `chunk_id` is in the top-`k` retrieved chunks.
 - `test_rag_answer_rejects_short_question` — empty `question` (length 0) returns 422.
 - `test_rag_answer_rejects_bad_k` — `k` outside `[1, 10]` (e.g. `k=99`) returns 422.

Task 4 — Pydantic models

All request and response shapes live in `api/models.py`. Use Pydantic v2 idioms (`model_dump()`, `model_validate()`, `model_config = ConfigDict(...)`); v1 idioms (`.dict()`, `parse_obj()`, `class Config:`) are deprecated and must not appear under `api/`.

The minimum shapes (the reading covers them in detail in Sections 5–7):

- `ExtractRequest`, `Entity`, `ExtractResponse`
- `KGRequest`, `KGResponse`, `UnsupportedQueryDetail` (the structured 422 detail body for `/kg/query`)
- `RAGRequest`, `Citation`, `RAGResponse`
- `HealthResponse` (for `/healthz`) and `ReadyDetail` (for `/readyz`'s 200 + 503 detail bodies)

Task 5 — FastAPI infrastructure

- `api/main.py` — the `FastAPI(lifespan=...)` app, the three path operations, the `/healthz` and `/readyz` endpoints, the CORS middleware. A request-id logging middleware (one-line `JSON` log per request with `request_id`, `path`, `method`, `status`, `latency_ms`) is **recommended** practice — add it if you want the observability hooks for Module 11; it is not a required deliverable.
- `/healthz` — process-level. Returns 200 even if Neo4j or Weaviate are down. Tests verify this — stopping the data tiers must not flip `/healthz`.
- `/readyz` — dependency-level. Probes Neo4j with `RETURN 1 AS ok` and Weaviate with `client.is_ready()`; returns 503 with structured detail if either fails; returns 200 only if both pass. Tests stop one backend at a time and verify the flip.
- CORS** — `CORSMiddleware` allowing the `WEB_ORIGIN` request (defaulting to `http://localhost:3000`). A browser preflight `OPTIONS` must receive the `Access-Control-Allow-Origin` header.
- `api/Dockerfile` — single-stage `python:3.11-slim`, cache-friendly layer order. Build context is the **repo root** (parent of `api/`); the Dockerfile copies `api/` as a package into `/app/api/` and runs `CMD ["python", "api/main:app", "--port", "8000", "0.0.0.0", "--port", "8000"]`. This preserves the package-relative imports (`from deps import ...`) — `cd api && uvicorn main:app` from inside `/app/api` would break them with `ImportError: attempted relative import with no known parent package`. Author a matching `.dockerignore`.
- Lab Docker Compose** (`docker-compose.yml` at the repo root) — two services only: `api` (build context `.`, with `dockerfile: api/Dockerfile`) and `web` (built from `/web`, multi-stage). Healthchecks on both; `web` depends on `api` healthy. **Do not write** `build: ./api` — the container will crash at startup with `ModuleNotFoundError: No module named 'api'`. Use the long-form, and declare `NEXT_PUBLIC_API_URL` as a **build arg** under `web.build.args` (NextJS bakes `NEXT_PUBLIC_*` into the client bundle at build time — a runtime `environment`; `env` runs in the container but never reaches the browser):

```
api:
  build:
    context: .
    dockerfile: api/Dockerfile
web:
  build:
    context: ./web
  args:
    NEXT_PUBLIC_API_URL: "http://localhost:8000"
```

Neo4j and Weaviate are external for the Lab — you run them on the host directly (the Lab README documents this). The Integration adds them to Compose.

Task 6 — Next.js frontend

The `web/` directory ships scaffolded — `package.json`, `package-lock.json`, `tsconfig.json`, `next.config.js`, `_app.tsx`, `pages/index.tsx` (landing), `styles/global.css`, `.dockerignore`. You author:

- `web/pages/extract.tsx` — text input → `POST /extract` → render entities with span highlights
- `web/pages/kg.tsx` — question input → `POST /kg/query` → render the Cypher string and a table of rows.
- `web/pages/rag.tsx` — question input → `POST /rag/answer` → render the answer with inline `[N]` style citation markers. Each marker must carry `data-testid="citation-marker"` so the Playwright smoke test can click it. Making the markers clickable or hoverable to reveal the source chunk text is a recommended UX polish; it is not a required deliverable.
- `web/lib/types.ts` — exactly three TypeScript interfaces mirroring `ExtractRequest`, `KGResponse`, `RAGResponse`. **Exactly. Field names must match the Pydantic shapes character-for-character** — `chunk_id` not `chunksId`, `start` not `startChar`. A drift between the Pydantic shape and the TS interface produces an empty page render that fails silently.
- `web/Dockerfile` — multi-stage `node:20-alpine`. The builder runs `apk add --no-cache libc6-compat` (so the `@next/swc-linux-x64-gnu` binary locked in `package-lock.json` loads on alpine's musl libc), then `npm ci && npm run build`. The runner ships only the `standalone` server bundle — `next.config.js` sets output: `'standalone'`, which writes a **self-contained** server under `next/standalone/` that the runner can serve without `node_modules`. Copy `next/standalone/`, `next/static/`, and `public/` into the runner, `CMD ["node", "server.js"]`. Image-size target: < **400 MB** — the `standalone` runner lands around 150 MB; a non-standalone runner with `npm ci --omit=dev` runs ~640 MB and a full `node_modules` single-stage build ~700 MB. Aim for the `standalone` shape.
- `web/.env.local.example` — `NEXT_PUBLIC_API_URL=http://localhost:8000`. The Lab repo ships this as `.env.local.example`; copy to `.env.local` locally. **Do not set** this to `http://api:8000` — the browser running on your laptop cannot reach `Compose` service-name DNS.

Three error states the pages must handle:

- 422** — show the `detail` field (Pydantic validation errors or `UnsupportedQueryError` structured detail).
- 503** — show “The backend is starting up — please try again in a moment.”
- Network error** — show “Could not reach the backend.”

Running locally

```
# In one terminal - start Neo4j and Weaviate on the host
docker run -d --name m10-neo4j -p 7687:7687 -p 7474:7474 \
-e NEO4J_AUTH=neo4j/password \
-e NEO4J_dbms_memory_heap_max_size=1G \
-e NEO4J_dbms_memory_pagecache_size=512M \
neo4j:5-community

docker run -d --name m10-weaviate -p 8080:8080 \
-e QUERY_DEFAULTS_LIMIT=20 \
-e AUTHENTICATION_ANONYMOUS_ACCESS_ENABLED=true \
-e PERSISTENCE_DATA_PATH=/var/lib/weaviate \
-e DEFAULT_VECTORIZER_MODULE=none \
semitechnologies/weaviate:1.24.10

# Seed Neo4j by piping seed.cypher into the running container - no
# host-side cypher-shell install required (the neo4j:5-community image
# ships with it). The Neo4j browser at http://localhost:7474 is the GUI
# alternative if you prefer to paste the seed by hand.
docker exec -i m10-neo4j cypher-shell -u neo4j -p password < api/seed.cypher

# Seed Weaviate from your venv (sentence-transformers must be installed -
# see the Reading's Setup section).
cd api
python seed_weaviate.py
cd ..

# Bring up the Lab Compose (api + web). First create api/.env from
# api/.env.example - Compose reads env_file relative to the compose
# file, not the service build context.
cp api/.env.example api/.env # edit values as needed
docker compose up -d --build
open http://localhost:3000
```

Or run the backend and frontend without Docker for iteration. The starter `api/` files use **package-relative imports** (e.g., `from .models import Entity`), so `uvicorn` must be launched from the **repo root** with the dotted path `api.main:app` — never `cd api && uvicorn main:app`, which makes `main` a top-level module and breaks the relative imports (`ImportError: attempted relative import with no known parent package`. Author the Dockerfile's `CMD` the same way (`uvicorn api.main:app ...` with the working directory set to the repo root).

Tests

Run the full Lab suite:

```
# Backend
pytest tests/backend -v

# Frontend
cd web && npm run build
cd ..
npm playwright install --with-deps chromium
npm playwright test tests/frontend/playwright

# Dockerfile smoke
docker build -t m10-api -f api/Dockerfile .
docker build -t m10-web web/
```

CI runs the full matrix: Python 3.11 + Node 20, services for Neo4j + Weaviate, seed scripts, `pytest`, `npm` run build; Playwright headless assertions against your rendered pages.

Challenge Extensions

- Tier 1 — OpenAPI-driven TypeScript client**
- Replace the synchronous path operations (`web/lib/types.ts` with codegen output from the FastAPI OpenAPI schema. Use `interfaces=typescript` (or `openapi=typescript-codegen`). Pages import the generated client and call methods like `api.extract(text)` instead of raw `fetch`. Substantive because it introduces contract-driven client generation — a real production pattern — and forces engagement with the OpenAPI schema the backend already exposes.
- Tier 2 — Async path operations + streaming RAG**
- Refactor the synchronous path operations to `async def`. Switch the Neo4j driver to `AsyncGraphDatabase` and the Weaviate client to its async API. Convert `/rag/answer` to a streaming endpoint emitting partial generated tokens via Server-Sent Events (SSE) as `flan-t5-base` produces them. Consume the stream in the Next.js RAG page with incremental rendering. Substantive because it introduces async-Python at the API surface, SSE, browser-side stream consumption, and incremental UI rendering — all real-world RAG patterns.
- Tier 3 — Reranking + multi-hop RAG**

Extend the RAG endpoint with (a) a cross-encoder reranker (`cross-encoder/ms-marco-MiniLM-L-6-v2` — carry-forward from Module 8) that reranks the top-20 Weaviate results down to top-4 by relevance; (b) a multi-hop retrieval pattern where the first answer's citations are used as queries for a follow-up retrieval if first-pass confidence is below 0.6; (c) confidence scoring that incorporates reranker scores. Ship a Markdown table with retrieval quality metrics on a held-out question set, comparing reranking on/off. Substantive: real production RAG engineering.

Submission

- Commit your changes to the `lab-10-fastapi-kg-service` branch and push.
- Open a PR against the `main` branch of your fork.
- Confirm the CI workflow succeeds.

Your PR description must include:

- A brief description of your approach.
- Any deviations from the starter.
- Confirmation that `docker build` succeeds for both `api/Dockerfile` and `web/Dockerfile`, and `docker compose up -d` brings the two-services Lab stack up healthy.
- A screenshot of the Next.js `/rag` page rendering a cited answer to a seeded question.
- Paste your PR URL into TalentLMS —> Module 10 —> Lab 10 to submit this assignment.

Common pitfalls

- Loading `flan-t5-base` per request** instead of in `lifespan` — cold-start hangs and intermittent test timeouts.
- Returning the generator's raw output without citation extraction** — ungrounded answers must return the sentinel response.
- Creating the Neo4j or Weaviate client at module import time** — clients are constructed before the network is up in CI, producing flaky failures.
- `NEXT_PUBLIC_API_URL=http://api:8000` — the browser cannot resolve `api`. Use `http://localhost:8000`.
- TypeScript interface drift** — Backend returns `chunk_id`; you wrote `chunkId` in `lib/types.ts`. The page renders empty.
- Forgetting `CORSMiddleware`** — the browser console shows the CORS error; curl works.
- Returning Neo4j records** — objects from `/kg/query` — JSON serialization fails. Convert with `record.data()`.
- Single-stage Next.js Dockerfile** — image balloons to ~700 MB; the multi-stage `standalone` runner lands around 150 MB.

